Análisis y diseño de algoritmos

Practica 2

Solución de problemas

Instrucciones

Resuelve el siguiente problema, emplea dos lenguajes de programación.

Tienes un pipeline que recibe una lista de registros de transacciones. Cada registro forma un diccionario con valores transaction_id y una amount. Debido a problemas en el proceso, pueden aparecer IDs duplicados en la lista.

Escribe una función que reciba la lista y regrese una una nueva lista sin duplicados, manteniendo solamente la primera aparición de cada transacción.

```
#Input
transactions = [
    {"transaction_id": "T1", "amount": 100},
    {"transaction_id": "T2", "amount": 200},
    {"transaction_id": "T1", "amount": 100},  # duplicate
    {"transaction_id": "T3", "amount": 300},
    {"transaction_id": "T2", "amount": 999},  # duplicate, different amount
]
# Expected output:
[
    {"transaction_id": "T1", "amount": 100},
    {"transaction_id": "T2", "amount": 200},
    {"transaction_id": "T3", "amount": 300},
]
```

Agrega también la complejidad tentativa de la memoria que consumiría tu solucion.

¿Qué otras cosas debería contemplar tu diseño?

-Inicio

Para resolver el problema planteado, se creó un algoritmo para revisar y eliminar los elementos duplicados de una lista de datos; el archivo en Excel o .json se usó como ejemplo para esta práctica.

Usando Python, importamos la librería Pandas, que nos facilita la extracción de los datos de un archivo como Excel al entorno de Python. Teniendo la librería, creamos la función “read_data”, que es la función que leerá todos los datos de nuestro archivo y nos devolverá esos mismos datos.

Después, se creó la función llamada “elim_dup”; lo que hace esta función es eliminar todos los datos que estén duplicados y nos entrega una librería limpia de datos.

La función “elim_dup” usa solo de referencia la ID. Usando un ciclo *for*, se usa una condición la cual nos indica que, si la ID que tenemos en esa iteración no está registrada, se agregará a la lista de las ID ya vistas y la fila entera se agregará a nuestra lista de datos final de forma ordenada. Si la condición no se cumple, solo se ignorará esa ID, dando a entender que esta ID ya se leyó anteriormente, así evitando tener datos duplicados.

Al final, se entrega en un archivo en Excel las dos listas de datos: la lista original y la lista sin datos duplicados. La lista que tenemos como ejemplo tiene alrededor de 5000 datos; al final del algoritmo pasa a menos de 4100 datos, dándonos a entender que se eliminaron de forma satisfactoria los datos duplicados.

```python
import pandas as pd

datos = 'transaction_data.xlsx'

def read_data():
    df = pd.read_excel(datos, sheet_name='transaction_data')
    return df
data = read_data()

def elim_dup(data):
    lista_transacciones = data.to_dict(orient='records')

    ids_vistos= set()
    data_sin_dup = []

    for fila in lista_transacciones:
        id_actual = fila['transaction_id']
        if id_actual not in ids_vistos:
            ids_vistos.add(id_actual)
            data_sin_dup.append(fila)
    return data_sin_dup

data_sin_dup = elim_dup(data)

df_limpio = pd.DataFrame(data_sin_dup)

nombre_archivo = 'comparativo_for.xlsx'

with pd.ExcelWriter(nombre_archivo) as writer:

    data.to_excel(writer, sheet_name='Datos_Originales', index=False)

    df_limpio.to_excel(writer, sheet_name='Sin_Duplicados', index=False)

print(f"Se creó el archivo '{nombre_archivo}' para la comparativa")
```

Excel de los últimos resultados de la lista original:

4985	T4112	880
4986	T4113	916
4987	T4114	280
4988	T2234	125
4989	T182	888
4990	T4115	383
4991	T4116	793
4992	T3988	847
4993	T4117	675
4994	T4118	503
4995	T4119	20
4996	T4120	641
4997	T2586	416
4998	T4047	175
4999	T2416	759
5000	T4121	903
5001	T4122	348

Excel de los últimos resultados de la lista después del uso del algoritmo:

4069	T4105	854
4070	T4106	313
4071	T4107	897
4072	T4108	163
4073	T4109	938
4074	T4110	78
4075	T4111	483
4076	T4112	880
4077	T4113	916
4078	T4114	280
4079	T4115	383
4080	T4116	793
4081	T4117	675
4082	T4118	503
4083	T4119	20
4084	T4120	641
4085	T4121	903
4086	T4122	348

-Complejidad

El tiempo que tarda el algoritmo crece de forma proporcional a la cantidad de datos. Si tienes 1000 filas, hace aproximadamente 1000 operaciones; si tienes un millón, hace un millón.

El ciclo for recorre el ciclo una vez lo que toma O(N), usando la función set() Python ya sabe si hay elementos duplicados o no, entonces la complejidad de este algoritmo seria O(N)

La complejidad afecta directamente a la memoria RAM usada, si el archivo de datos es grande entonces se usará bastante memoria

¿Qué otras cosas debería contemplar tu diseño?

Se podría agregar una condicional al final de la ejecución el cual evalúe si hay elementos duplicados en la lista de datos, usando la función set y solo tomando en cuenta la id de transacción se podría hacer esta condicional para evitar ejecutar el algoritmo de forma innecesaria.

```
if len(data['transaction_id']) == len(set(data['transaction_id'])):
    print("El archivo NO tiene duplicados")
    df_limpio = data.copy()
else:
    print("Se encontraron duplicados")
    data_sin_dup = elim_dup(data)
    df_limpio = pd.DataFrame(data_sin_dup)

nombre_archivo = 'comparativo_for.xlsx'
```

-Algoritmo final

```python
import pandas as pd

datos = 'transaction_data.xlsx'

def read_data():
    df = pd.read_excel(datos, sheet_name='transaction_data')
    return df
data = read_data()

def elim_dup(data):
    lista_transacciones = data.to_dict(orient='records')

    ids_vistos= set()
    data_sin_dup = []

    for fila in lista_transacciones:
        id_actual = fila['transaction_id']
        if id_actual not in ids_vistos:
            ids_vistos.add(id_actual)
            data_sin_dup.append(fila)
    return data_sin_dup

data_sin_dup = elim_dup(data)


if len(data['transaction_id']) == len(set(data['transaction_id])):
    print("El archivo NO tiene duplicados")
    df_limpio = data.copy()
else:
    print("Se encontraron duplicados")
    data_sin_dup = elim_dup(data)
    df_limpio = pd.DataFrame(data_sin_dup)

nombre_archivo = 'comparativo_for.xlsx'

with pd.ExcelWriter(nombre_archivo) as writer:

    data.to_excel(writer, sheet_name='Datos_Originales', index=False)

    df_limpio.to_excel(writer, sheet_name='Sin_Duplicados', index=False)

print(f"Se creó el archivo '{nombre_archivo}' para la comparativa")
```

Usando el código C actúa como un filtro donde entra un archivo de datos llamado, “transaction_data”, el cual se usa como información original para luego arrojar uno llamado “sin_duplicados”, donde se escriben los registros únicos, en caso de que uno de los archivos no pueda abrirse o genere un problema manda un error de que no se puede abrir los archivos, para el “borrado de ID” lo que hace es que guarda en un arreglo los ID y va verificando, si ya existe el ID lo ignora y pasa al siguiente, si es nuevo el ID lo guarda y pasa al siguiente, al final en la consola muestra si se encontraron duplicados o si ya estaba sin duplicados, aunque no trabaja con archivos directos de xlsx por temas de compatibilidad, pero maneja .CSV que son archivos prácticamente iguales de Excel,

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINEA 1024
#define MAX_TRANSACCIONES 5000

int main() {
    FILE *archivo_entrada = fopen("transaction_data.csv", "r");
    FILE *archivo_salida = fopen("sin_duplicados.csv", "w");

    if (archivo_entrada == NULL || archivo_salida == NULL) {
        printf("Error: No se pudieron abrir los archivos.\n");
        return 1;
    }

    char linea[MAX_LINEA];
    char ids_vistos[MAX_TRANSACCIONES][50];
    int num_vistos = 0;
    int hay_duplicados = 0;

    if (fgets(linea, MAX_LINEA, archivo_entrada)) {
        fprintf(archivo_salida, "%s", linea);
    }

    while (fgets(linea, MAX_LINEA, archivo_entrada)) {
        char copia_linea[MAX_LINEA];
        strcpy(copia_linea, linea);

        char *id_actual = strtok(copia_linea, ",");

        if (id_actual != NULL) {
            int es_duplicado = 0;

            for (int i = 0; i < num_vistos; i++) {
                if (strcmp(ids_vistos[i], id_actual) == 0) {
                    es_duplicado = 1;
                    hay_duplicados = 1;
                    break;
                }
            }

            if (!es_duplicado) {
                strcpy(ids_vistos[num_vistos], id_actual);
                num_vistos++;

                fprintf(archivo_salida, "%s", linea);
            }
        }
    }

    if (!hay_duplicados) {
        printf("El archivo NO tiene duplicados.\n");
    } else {
        printf("Se encontraron duplicados. Se filtraron correctamente.\n");
    }

    printf("Se creó el archivo 'sin_duplicados.csv' para la comparativa.\n");

    fclose(archivo_entrada);
    fclose(archivo_salida);

    return 0;
}
```

FOTO DEL FINAL DEL ARCHIVO “transaction_data” O ANTES DE EJECUTAR EL ALGORITMO

4984	T4111	483
4985	T4112	880
4986	T4113	916
4987	T4114	280
4988	T2234	125
4989	T182	888
4990	T4115	383
4991	T4116	793
4992	T3988	847
4993	T4117	675
4994	T4118	503
4995	T4119	20
4996	T4120	641
4997	T2586	416
4998	T4047	175
4999	T2416	759
5000	T4121	903
5001	T4122	348
5002		

FOTO DEL FINAL DEL ARCHIVO “sin_duplicados” O DESPUES DE EJECUTAR EL ALGORITMO

4069	T4105	854
4070	T4106	313
4071	T4107	897
4072	T4108	163
4073	T4109	938
4074	T4110	78
4075	T4111	483
4076	T4112	880
4077	T4113	916
4078	T4114	280
4079	T4115	383
4080	T4116	793
4081	T4117	675
4082	T4118	503
4083	T4119	20
4084	T4120	641
4085	T4121	903
4086	T4122	348
4087		

COMPLEJIDAD:

La complejidad de tiempo del algoritmo es O(nxm) donde “n” representa el número de líneas en el archivo, y “m” el número de registros únicos en el archivo, con la complejidad de memoria va O(rxp) (use otras letras para no confundir con las anteriores) donde r es el limite de ID, y p es el limite de caracteres que tiene cada ID, y la complejidad de comparación, es O(n) debido a que revisa letra por letra cada ID en un ciclo continuo donde “n” representa lo mismo que “p” en complejidad de memoria

Integrantes del equipo:

-Blancas Ibarra Yahel

-Rosillo Valencia Jared

-Iker Daniel Farias Sánchez

-Emmanuel Omar Soto Reyes

-Oscar Manuel

Repositorio: https://github.com/Yaimerx/ADA-Practica-2